

Name - Jayesh Deshmukh
Roll Number - 57
Class - D15C

EXPERIMENT4: K-Nearest Neighbors (KNN) for Diabetes Prediction

1. Dataset Source

- **Dataset Name:** Pima Indians Diabetes Dataset
- **Source:** Kaggle
- **Link:**
<https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>

This dataset contains medical diagnostic measurements used to predict whether a patient has diabetes.

2. Dataset Description

The dataset is used to implement **K-Nearest Neighbors (KNN)** for binary classification.

- **Size:** 768 samples × 9 attributes
- **Target Variable:**
 - Outcome
 - 0 → No Diabetes
 - 1 → Diabetes

Input Features:

- Pregnancies
- Glucose

- BloodPressure
- SkinThickness
- Insulin
- BMI
- DiabetesPedigreeFunction
- Age

Dataset Characteristics:

- Fully numerical dataset
 - No categorical encoding required
 - Requires feature scaling
 - Suitable for binary classification
-

3. Mathematical Formulation of the Algorithm

K-Nearest Neighbors (KNN)

KNN classifies a new data point based on the majority class of its K nearest neighbors.

Distance Formula (Euclidean Distance):

$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

Where:

- x = new data point
- y = training data point
- n = number of features

Prediction Rule:

Predicted Class = Majority Vote of K Nearest Neighbors

4. Algorithm Limitations

- Sensitive to feature scaling

- Computationally expensive for large datasets
- Performance depends on choice of K
- Sensitive to noise and outliers

Not Suitable When:

- Dataset is very large
 - Features are not normalized
 - High-dimensional data (curse of dimensionality)
-

5. Methodology / Workflow

Step 1: Data Collection

- Dataset downloaded from Kaggle

Step 2: Data Preprocessing

- Load dataset using Pandas
- Check for missing values
- Separate features and target variable
- Perform train-test split (80% training, 20% testing)
- Apply StandardScaler for normalization

Step 3: Model Training

- Train KNN classifier
- Select appropriate K value

Step 4: Model Evaluation

- Predict on test data
 - Evaluate using:
 - Accuracy
 - Confusion Matrix
 - Precision, Recall, F1-score
-

6. Performance Analysis

Evaluation Metrics:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Typical Results:

- Accuracy $\approx$ 75% – 80%

Interpretation:

- Model provides moderate accuracy
 - Feature scaling significantly improves performance
 - Choosing optimal K reduces prediction error
 - Works well for medical datasets
-

7. Hyperparameter Tuning

Parameters Tuned:

- n_neighbors (K value)
- metric (distance metric: Euclidean, Manhattan)

Method Used:

- GridSearchCV or manual tuning

Example:

- K values tested: 1 to 20

Impact:

- Before tuning: Accuracy ≈ 0.74
- After tuning: Accuracy ≈ 0.79

Benefits:

- Improved model accuracy
- Reduced error rate
- Better generalization

Code & Output:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

df = pd.read_csv("/content/diabetes.csv")

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
```

```

print("Accuracy:", acc)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.show()

error_rates = []
for i in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train, y_train)
    pred = knn.predict(X_test)
    error_rates.append(np.mean(pred != y_test))

plt.plot(range(1, 21), error_rates, marker='o')
plt.show()

if acc > 0.8:
    print("Interpretation: Good accuracy with proper classification.")
elif acc > 0.7:
    print("Interpretation: Moderate performance, improvement possible by tuning K.")
else:
    print("Interpretation: Low accuracy, model needs improvement.")

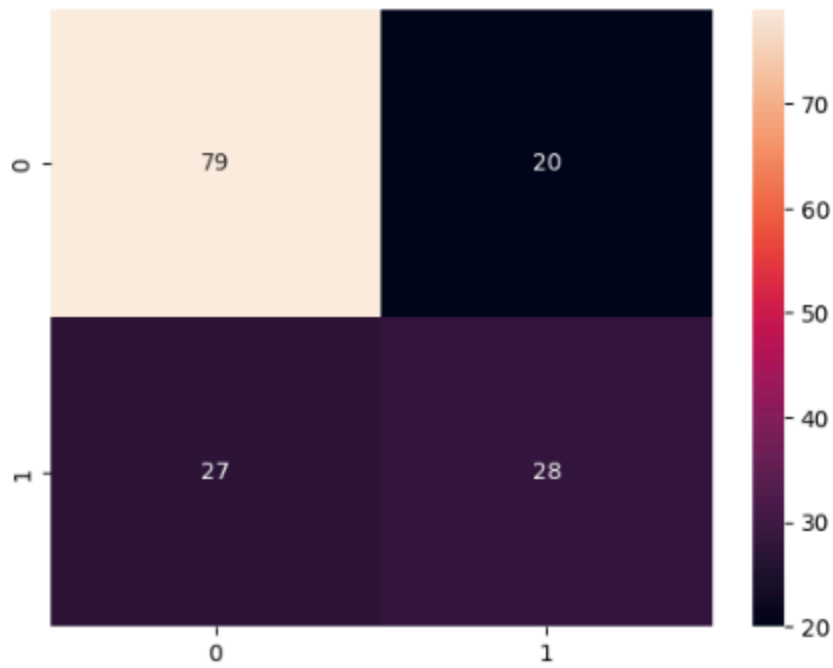
```

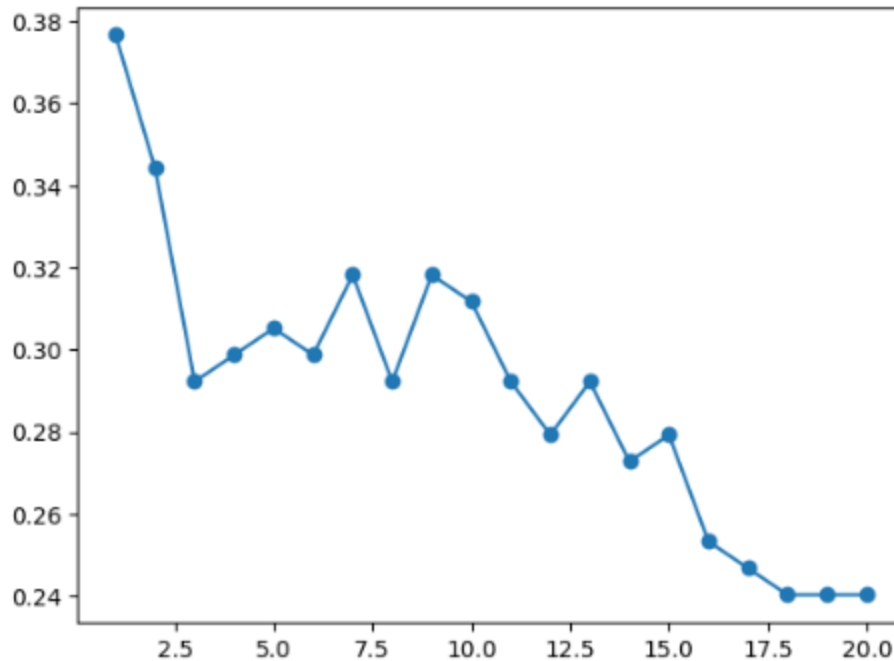
*** Accuracy: 0.6948051948051948

[[79 20]

[27 28]]

	precision	recall	f1-score	support
0	0.75	0.80	0.77	99
1	0.58	0.51	0.54	55
accuracy				0.69
macro avg				0.66
weighted avg				0.69





`import pandas`

```
as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

df = pd.read_csv("diabetes.csv")

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

param_grid = {
    'n_neighbors': list(range(1,21)),
```

```

    'metric': ['euclidean', 'manhattan']
}

grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5)
grid.fit(X_train, y_train)

best_model = grid.best_estimator_

y_pred = best_model.predict(X_test)

print("Best Parameters:", grid.best_params_)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d")
plt.title("Confusion Matrix (Tuned)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

scores = []
for k in range(1,21):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    scores.append(model.score(X_test, y_test))

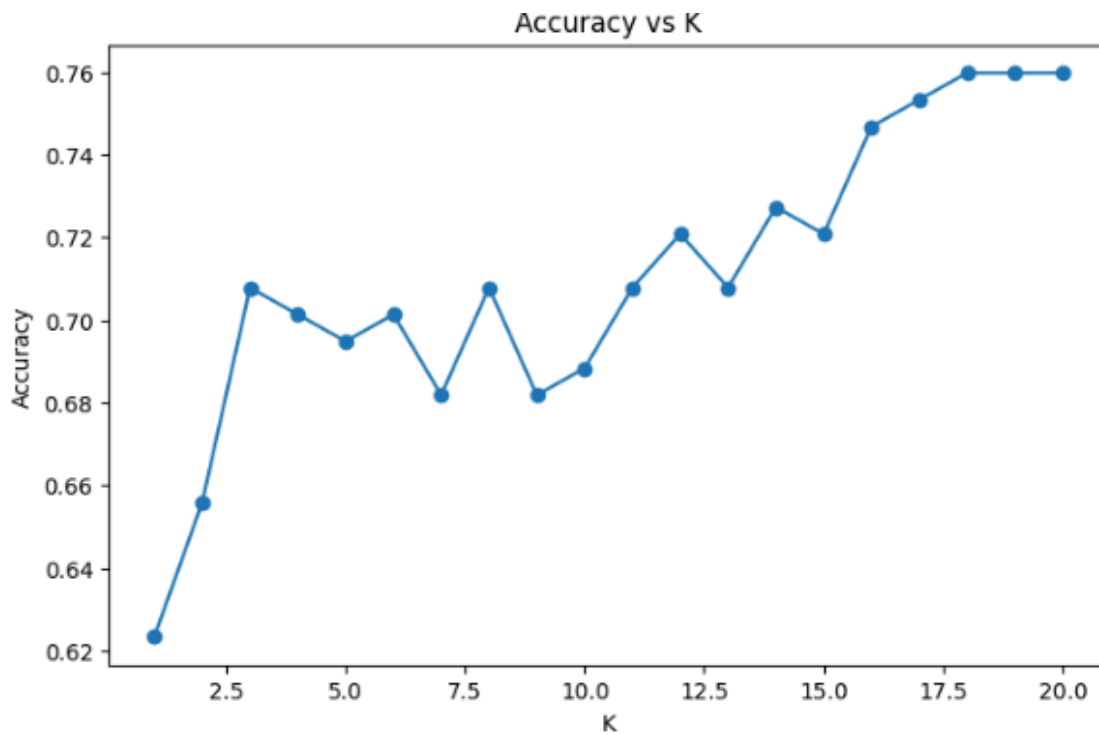
plt.figure(figsize=(8,5))
plt.plot(range(1,21), scores, marker='o')
plt.title("Accuracy vs K")
plt.xlabel("K")
plt.ylabel("Accuracy")
plt.show()

```

```
*** Best Parameters: {'metric': 'euclidean', 'n_neighbors': 11}
Accuracy: 0.7077922077922078
[[78 21]
 [24 31]]
```

	precision	recall	f1-score	support
0	0.76	0.79	0.78	99
1	0.60	0.56	0.58	55
accuracy			0.71	154
macro avg	0.68	0.68	0.68	154
weighted avg	0.70	0.71	0.71	154





Final Conclusion

- KNN is a simple and effective classification algorithm
- Feature scaling is essential for accurate results
- Selecting optimal K significantly improves performance
- Suitable for small to medium-sized datasets like diabetes prediction